

Módulo 5. CI/CD para ML, monitorización y detección de drift

Curso MLOps/DataOps · ANBAN

José Picón

2026

1. Las tres siglas: CI, CD y CT

En MLOps se usan tres siglas que suelen confundirse:

- **CI (Continuous Integration)**. Cada commit pasa por test, lint y build automáticos. Garantiza que el repositorio no se rompe.
- **CD (Continuous Delivery o Continuous Deployment)**. La salida del CI se puede desplegar a producción con un clic (delivery) o automáticamente (deployment).
- **CT (Continuous Training)**. Específico de ML. Reentrenamiento automático del modelo cuando llegan datos nuevos o cuando se detecta drift.

En un sistema ML maduro corren los tres bucles:

```
commit → CI → test + train + eval + register
                                     ↓
datos nuevos → CT → reentrena → eval vs prod
                                     ↓
schedule → CD → despliega si mejora
```

El de la izquierda es el de DevOps. Los otros dos son los que añade MLOps.

2. Estrategias de despliegue

Cuando promocionas un modelo nuevo a producción, hay tres estrategias clásicas para hacerlo sin riesgo:

Estrategia	Cómo funciona	Cuándo elegirla
Blue-green	Dos entornos paralelos; cambias el tráfico de uno a otro	Crítico, necesitas rollback en segundos
Canary	Empiezas con 5 % del tráfico, vas subiendo hasta 100 %	Detectar bugs sin daño masivo
Shadow	El modelo nuevo recibe el tráfico real pero no responde al cliente; comparas con el viejo	Validar el nuevo durante días antes de cambiar

En ML, la estrategia **shadow** es valiosísima: te permite ejecutar el modelo nuevo durante una semana con tráfico real antes de sustituir nada.

3. Monitorización: dos familias de métricas

Cuando un modelo está en producción, tienes que medirlo en dos planos distintos:

3.1 Métricas de servicio

Son las de cualquier API:

- Latencia (p50, p95, p99).
- Throughput (peticiones por segundo).
- Tasa de errores (5xx, 4xx).
- Uso de CPU y memoria del contenedor.

Estas las recoges con Prometheus + Grafana (o con servicios cloud equivalentes como CloudWatch, Data-dog, New Relic).

3.2 Métricas de modelo

Son específicas de ML:

- **Distribución de outputs.** Si el modelo ahora dice “sí” un 80 % de las veces en lugar del 20 % habitual, algo ha cambiado.
- **Accuracy proxy.** Cuando tardas en tener la etiqueta real (ej. “morosidad a 90 días”), usas un proxy hasta que llegue.
- **Drift de inputs.** La distribución de los datos que recibe el modelo cambia respecto a los datos con los que entrenó. Es el tema del próximo apartado.

4. Drift: la enfermedad silenciosa

Un modelo en producción puede dejar de funcionar **sin que se rompa nada técnico**. El código no falla, la API responde, las métricas de servicio están verdes. Pero las predicciones son cada vez peores. Eso es **drift**: la realidad ha cambiado y el modelo ya no la representa.

Hay tres tipos:

Tipo	Qué cambia	Ejemplo
Data drift (covariate shift)	$P(X)$, la distribución de inputs	Tras la pandemia, la edad media de los clientes online subió 10 años
Concept drift	$P(y X)$, la relación entre input y output	Lo que antes predecía morosidad ya no lo predice
Label drift	$P(y)$, la distribución de la etiqueta	Antes 20 % de morosos, ahora 35 %

4.1 Tests estadísticos para detectar drift

Cuatro tests habituales:

Test	Tipo de variable	Comentario
PSI	Numérica	Umbrales intuitivos: 0,1 estable, 0,1-0,25 vigilar, >0,25 drift
Kolmogorov-Smirnov (KS)	Numérica	Compara distribuciones, devuelve p-value
Chi-cuadrado	Categórica	Mide independencia entre dos categóricas
Wasserstein	Numérica	Distancia “earth mover” entre distribuciones

4.2 PSI explicado a fondo

PSI (Population Stability Index) es el más usado en industria por su simplicidad. Para una variable numérica el algoritmo es:

1. Divides la variable en N bins (típicamente 10 deciles del dataset de referencia).
2. Para cada bin, calculas la proporción de registros del dataset de referencia (`expected_i`) y del dataset actual (`actual_i`).

3. Sumas sobre todos los bins:

$$\text{PSI} = \sum_i (\text{actual}_i - \text{expected}_i) * \ln(\text{actual}_i / \text{expected}_i)$$

Interpretación operativa:

Valor de PSI	Significado
< 0,1	Distribución estable, sin acción
0,1 a 0,25	Cambio moderado, vigilar
≥ 0,25	Drift significativo, investigar y posiblemente reentrenar

5. Las librerías principales

5.1 Evidently

Evidently es la librería de código abierto más usada para monitorizar modelos. Te permite generar informes HTML interactivos comparando dos datasets (referencia vs actual) y detectar drift, calidad de datos y rendimiento.

Para instalarla:

```
pip install evidently
```

Es opcional: si no está instalada, nuestro script `drift_report.py` cae a un fallback que calcula PSI a mano y genera un HTML mínimo.

5.2 Numpy

NumPy es la base del cálculo numérico en Python. La usamos para crear histogramas y calcular cuantiles.

```
pip install numpy
```

Operaciones que aparecen en el script:

```
import numpy as np

# Cuantiles (para crear bins):
breaks = np.quantile(reference.dropna().to_numpy(), np.linspace(0, 1, 11))

# Histograma:
hist, _ = np.histogram(current.dropna(), bins=breaks)

# Operaciones elemento a elemento:
psi = np.sum((actual_pct - expected_pct) * np.log(actual_pct / expected_pct))
```

5.3 GitHub Actions

GitHub Actions no es una librería, es un servicio integrado en GitHub. Permite ejecutar workflows (pipelines de CI/CD) cada vez que ocurre un evento en tu repositorio (push, pull request, schedule, manual).

Un workflow es un fichero YAML en `.github/workflows/`. Ejemplo mínimo:

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest
```

6. Antes de la práctica: requisitos

Para hacer este lab necesitas:

1. **Lab 1 y Lab 2 completados.** El Lab 4 lee el test.parquet del Lab 1 y el modelo registrado del Lab 2.
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab4_cicd_monitoring/`.**

7. Ejercicio: detectar drift y promoción condicional

Objetivo del ejercicio: generar un dataset con drift inyectado, calcular el PSI por feature, ver el reporte HTML, y probar el mecanismo de promoción condicional que solo asciende un modelo si bate al actual de Production.

7.1 Paso 1 — Exportar variables de entorno

Las mismas que en el Lab 2:

```
export MLFLOW_TRACKING_URI=http://localhost:5050
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

7.2 Paso 2 — Generar un dataset con drift sintético

El script `synthetic/make_drift.py` toma el `test.parquet` del Lab 1 y le aplica cuatro modificaciones que simulan cambios clínicos plausibles:

- Suma 8 años a `age` (la población de pacientes envejece).
- Aumenta la prevalencia de `high_blood_pressure` en 10 %.
- Multiplica `creatinine_phosphokinase` por 1,5 (cambio de método de laboratorio).
- Cambia aleatoriamente el 4 % de las etiquetas `DEATH_EVENT` (leve cambio de prevalencia).

Ejecútalo:

```
mkdir -p data/processed reports
python synthetic/make_drift.py
```

Salida:

```
OK · data/processed/drifted.parquet (60 filas con drift inyectado)
```

7.3 Paso 3 — Generar el reporte de drift

El script `src/drift_report.py` compara dos parquet (referencia vs actual) y genera un HTML con el análisis. Lo usamos así:

```
python src/drift_report.py \  
  --reference ../lab1_dataops/data/processed/test.parquet \  
  --current data/processed/drifted.parquet \  
  --output reports/drift.html
```

Si Evidently está instalado, hace un análisis muy completo (gráficos de distribuciones, mapas de correlaciones, etc.). Si no, cae al fallback PSI que produce un HTML simple con una tabla.

Cuando termina, abre `reports/drift.html` en el navegador.

7.4 Paso 4 — Interpretar los resultados

Mira `reports/drift.json`. Es un diccionario `{feature: PSI}`.

```
cat reports/drift.json | head -20
```

Las features con `PSI > 0,25` son las que han cambiado mucho. Típicamente: `age`, `creatinine_phosphokinase`, `high_blood_pressure`.

Reflexión: si vieras este reporte en producción real, ¿qué harías? Tres opciones razonables:

- Reentrenar el modelo con datos recientes.
- Investigar de dónde viene el cambio (¿un bug en el ETL?, ¿cambio de mercado?, ¿error en la fuente?).
- Aceptar el drift si es esperado y no afecta a la métrica de negocio.

7.5 Paso 5 — La lógica de promoción condicional

El script `src/promote_if_better.py` automatiza la decisión: solo promociona un candidato a Producción si bate al actual en una métrica concreta, por un margen mínimo.

Léelo:

```
cat src/promote_if_better.py
```

La lógica clave:

```

candidate_metric = metric_for_version(client, name, candidate, args.metric)

prod_metric = metric_for_version(client, name, prod, args.metric)

threshold = prod_metric * (1 + args.min_improvement)
if candidate_metric >= threshold:
    client.transition_model_version_stage(name, candidate, "Production")

    print("PROMOTE")
else:
    print("REJECT")

```

El parámetro `--min-improvement 0.01` significa “solo promociona si mejora al menos un 1 %”. Esto evita que el sistema oscile entre modelos muy parecidos.

7.6 Paso 6 — Probarlo en local

Si aún no hay versión en Production, el script promociona la candidata automáticamente. Pruébalo:

```

python src/promote_if_better.py \
  --name heart-failure-clf \
  --metric f1 \
  --min-improvement 0.01

```

Salida típica:

```

candidate v1: f1=0.6877
no hay Production aún · promovemos candidato a Staging

```

7.7 Paso 7 — Revisar el workflow de GitHub Actions

El fichero `.github/workflows/ml.yml` define un workflow que en un repositorio real se ejecutaría en cada push. No vamos a ejecutarlo aquí (necesitas GitHub), pero conviene leerlo:

```

cat .github/workflows/ml.yml

```

Estructura habitual:

```

name: ml-pipeline
on:
  push:
    branches: [main]
  pull_request:
  schedule:
    - cron: "0 3 * * 0" # los domingos a las 3 AM
  workflow_dispatch: # botón "Run workflow" en la UI

jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps: ...

  train:
    needs: lint-test
    runs-on: ubuntu-latest
    steps: ...

  evaluate-and-promote:
    needs: train
    runs-on: ubuntu-latest
    steps: ...

  drift-report:
    schedule: "0 3 * * 0"
    steps: ...

```

Tres jobs encadenados con `needs:` (cada uno espera al anterior). El último corre solo en schedule (semanal) y genera el reporte de drift.

8. Checklist de cierre

- ¿Has generado el dataset `drifted.parquet` ?
- ¿Has generado `reports/drift.html` y lo has abierto en el navegador?
- ¿Has identificado al menos tres features con $PSI > 0,25$?
- ¿Has probado `promote_if_better.py` y has visto la decisión PROMOTE o REJECT?
- ¿Has leído el workflow `.github/workflows/ml.yml` y entiendes los jobs encadenados?

Si todo es sí, pasas al módulo 6 (governance y caso end-to-end).

9. Para profundizar

- **Evidently**: <https://docs.evidentlyai.com>. Documentación con ejemplos visuales.
- **GitHub Actions docs**: <https://docs.github.com/actions>.
- **Reliable Machine Learning**, Cathy Chen et al. (O'Reilly). Capítulos de monitoring.
- **NannyML**: <https://nannyml.readthedocs.io>. Estima accuracy cuando aún no tienes las etiquetas reales.